

AI Assisted Coding

Lab Assignment – 13.5

Name: K. Sai Krishna Reddy

HT.No: 2303A52305

Batch: 45

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.

- o Refactor the code to pass values using function parameters.

- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a

- configuration structure.

```
1 '''Task Description #1 (Refactoring & Removing Global Variables)
2 • Task: Use AI to eliminate unnecessary global variables from the
3 code.
4 • Instructions:
5 o Identify global variables used across functions.
6 o Refactor the code to pass values using function parameters.
7 o Improve modularity and testability.
8 • Sample Legacy Code:
9 rate = 0.1
10 def calculate_interest(amount):
11     return amount * rate
12     print(calculate_interest(1000))
13 • Expected Output:
14 o Refactored version passing rate as a parameter or using a
15 configuration structure.'''
16
17 # Refactored Code without Global Variables
18 def calculate_interest(amount, rate):
19     return amount * rate
20 # Example usage
21 interest_rate = 0.1
22 print(calculate_interest(1000, interest_rate))
23 # Alternatively, using a configuration structure
24 class Config:
25     def __init__(self, rate):
26         self.rate = rate
27     def calculate_interest_with_config(amount, config):
28         return amount * config.rate
29 # Example usage with configuration structure
30 config = Config(rate=0.1)
31 print(calculate_interest_with_config(1000, config))
32
```

100.0
100.0
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52163' '--' 'c:\3year-2sem\AI ASSISTANT CO
03A52305 AI A-C Ass-13.5\Task-1.py'
100.0
100.0
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas:

- o Readability

- o Logical simplification

- o Maintainability

Legacy Code:

score = 78

if score >= 90:

print("Excellent")

else:

if score >= 75:

```
print("Very Good")
```

```
else:
```

```
if score >= 60:
```

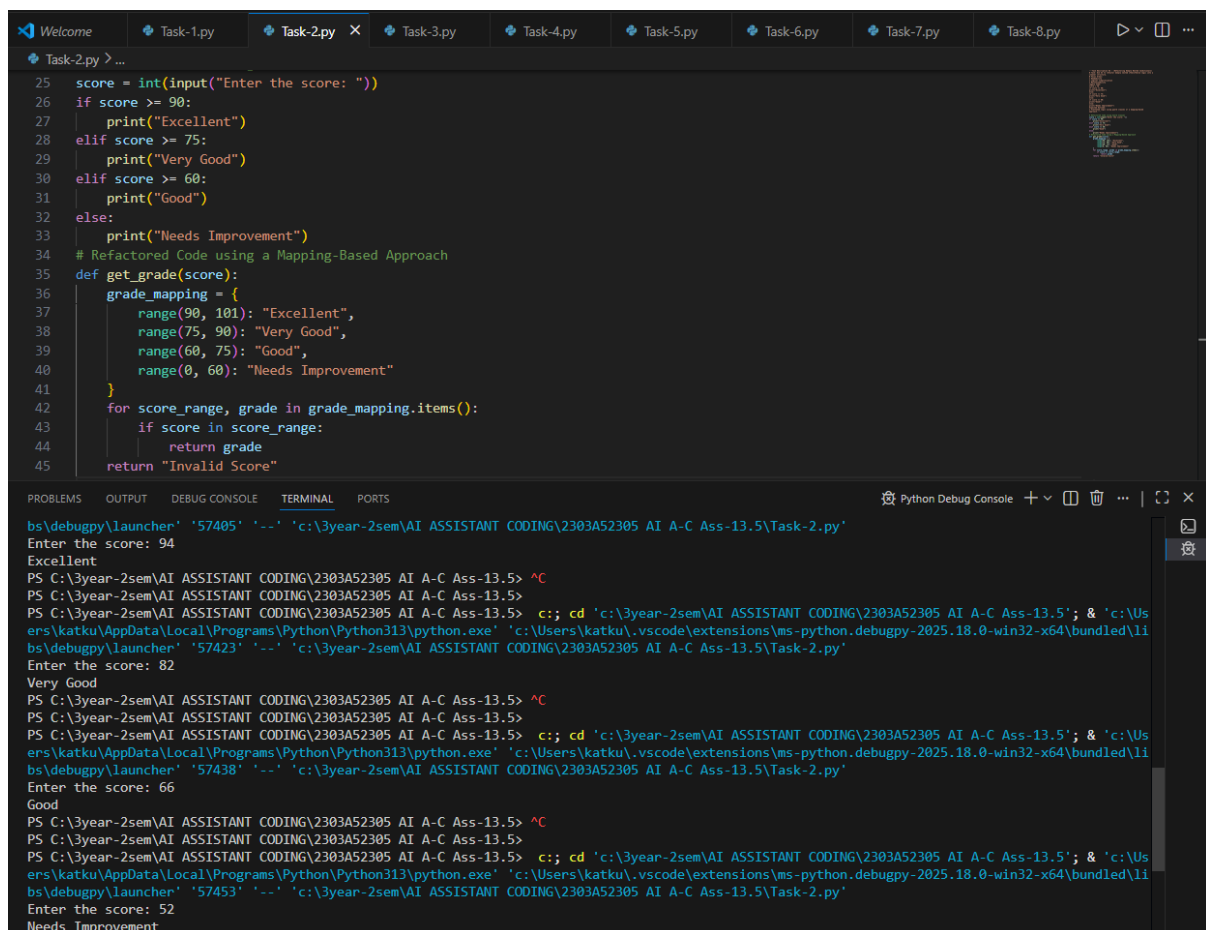
```
print("Good")
```

```
else:
```

```
print("Needs Improvement")
```

Expected Outcome:

o Flattened logic using guard clauses or a mapping-based approach.



The screenshot shows a VS Code editor with a Python file named 'Task-2.py'. The code defines a function 'get_grade(score)' that uses a mapping-based approach to determine the grade for a given score. The terminal output shows the execution of the script, with the user entering scores and the program outputting the corresponding grade.

```
25 score = int(input("Enter the score: "))
26 if score >= 90:
27     print("Excellent")
28 elif score >= 75:
29     print("Very Good")
30 elif score >= 60:
31     print("Good")
32 else:
33     print("Needs Improvement")
34 # Refactored Code using a Mapping-Based Approach
35 def get_grade(score):
36     grade_mapping = {
37         range(90, 101): "Excellent",
38         range(75, 90): "Very Good",
39         range(60, 75): "Good",
40         range(0, 60): "Needs Improvement"
41     }
42     for score_range, grade in grade_mapping.items():
43         if score in score_range:
44             return grade
45     return "Invalid Score"
```

Terminal Output:

```
bs\debugpy\launcher '57405' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-2.py'
Enter the score: 94
Excellent
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib\python313\python.exe' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-2.py'
Enter the score: 82
Very Good
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib\python313\python.exe' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-2.py'
Enter the score: 66
Good
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib\python313\python.exe' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-2.py'
Enter the score: 52
Needs Improvement
```

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.
- Focus Areas:

- o DRY principle
- o Context managers
- o Function reuse

Legacy Code:

```
f = open("data1.txt")
```

```
print(f.read())
```

```
f.close()
```

```
f = open("data2.txt")
```

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

- o Reusable function using with open() and parameters.

The screenshot shows a code editor with a dark theme. The top bar displays several open files: task1.py, task2.py, task3.py (active), data1.txt, and data2.txt. The main editor area shows a Python script with the following content:

```

9   f = open("data1.txt")
10  print(f.read())
11  f.close()
12  f = open("data2.txt")
13  print(f.read())
14  f.close()
15  Expected Outcome:
16  o Reusable function using with open() and parameters.
17  ...
18  # Refactored version using a reusable function with context managers.
19  import os
20
21  def read_file(filename):
22      with open(filename) as f:
23          print(f.read())
24
25  # Get the directory of the current script
26  script_dir = os.path.dirname(os.path.abspath(__file__))
27  read_file(os.path.join(script_dir, "data1.txt"))
28  read_file(os.path.join(script_dir, "data2.txt"))

```

Below the editor, the 'TERMINAL' tab is active, showing the command prompt output:

```

PS D:\AI_assited_coding> & C:/Users/vaish/AppData/Local/Programs/Python/Python314/python.exe d:/AI_assited_coding/assignment13_5/task3.py
hello world
hello world
PS D:\AI_assited_coding> & "d:\AI_assited_coding\assignment 8.5\.venv\Scripts\Activate.ps1"
(.venv) PS D:\AI_assited_coding>

```

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

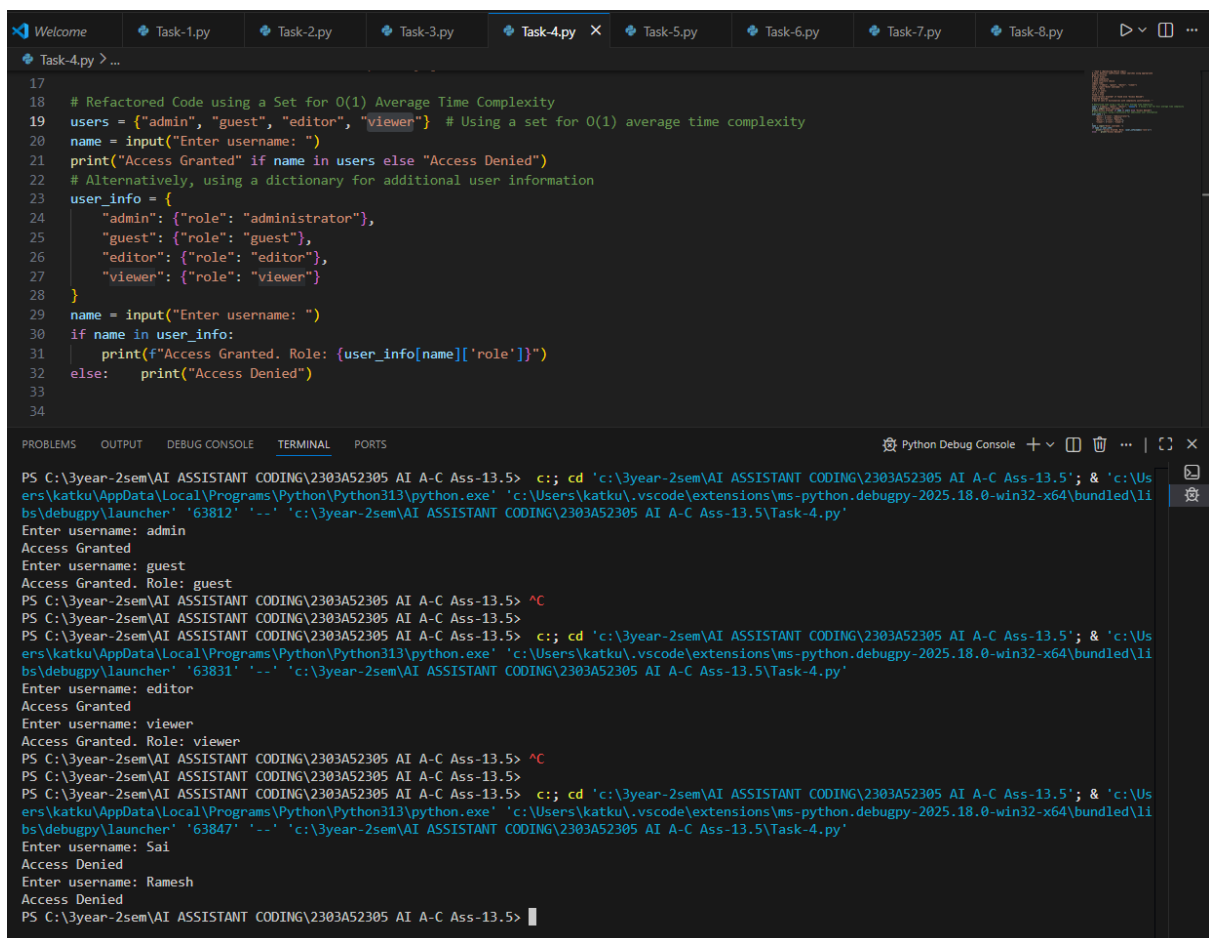
```
if u == name:
```

```
found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification.



The screenshot shows a VS Code editor with a file named 'Task-4.py' open. The code in the file is as follows:

```
17
18 # Refactored Code using a Set for O(1) Average Time Complexity
19 users = {"admin", "guest", "editor", "viewer"} # Using a set for O(1) average time complexity
20 name = input("Enter username: ")
21 print("Access Granted" if name in users else "Access Denied")
22 # Alternatively, using a dictionary for additional user information
23 user_info = {
24     "admin": {"role": "administrator"},
25     "guest": {"role": "guest"},
26     "editor": {"role": "editor"},
27     "viewer": {"role": "viewer"}
28 }
29 name = input("Enter username: ")
30 if name in user_info:
31     print(f"Access Granted. Role: {user_info[name]['role']}")
32 else:
33     print("Access Denied")
34
```

Below the code editor, the 'TERMINAL' tab is active, showing the execution of the script. The prompt is 'PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>'. The user enters 'admin', 'guest', 'editor', and 'viewer', all of which result in 'Access Granted' with their respective roles. The user then enters 'Sai' and 'Ramesh', both of which result in 'Access Denied'.

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

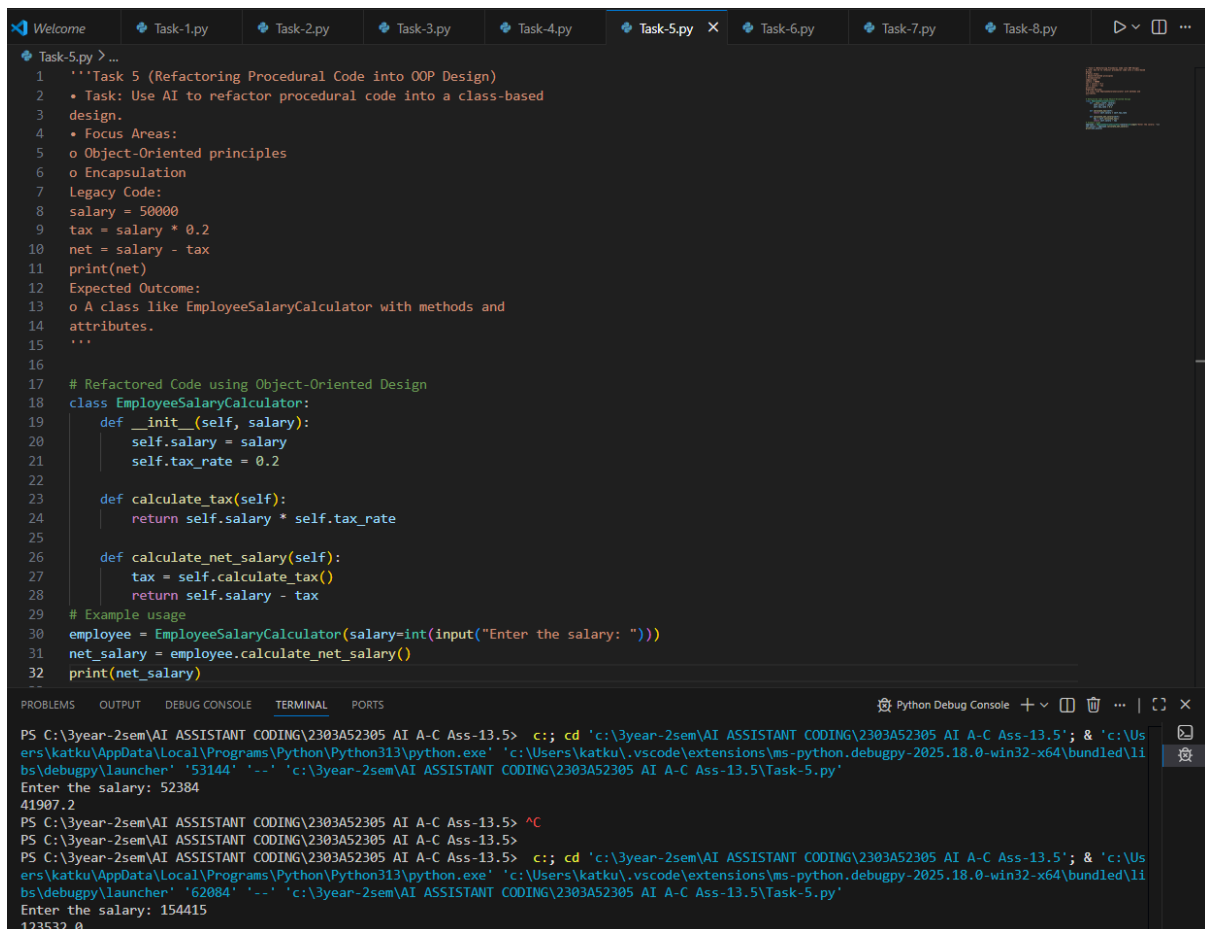
```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.



```
Task-5.py > ...
1  """Task 5 (Refactoring Procedural Code into OOP Design)
2  • Task: Use AI to refactor procedural code into a class-based
3  design.
4  • Focus Areas:
5  o Object-Oriented principles
6  o Encapsulation
7  Legacy Code:
8  salary = 50000
9  tax = salary * 0.2
10 net = salary - tax
11 print(net)
12 Expected Outcome:
13 o A class like EmployeeSalaryCalculator with methods and
14 attributes.
15 """
16
17 # Refactored Code using Object-Oriented Design
18 class EmployeeSalaryCalculator:
19     def __init__(self, salary):
20         self.salary = salary
21         self.tax_rate = 0.2
22
23     def calculate_tax(self):
24         return self.salary * self.tax_rate
25
26     def calculate_net_salary(self):
27         tax = self.calculate_tax()
28         return self.salary - tax
29
30 # Example usage
31 employee = EmployeeSalaryCalculator(salary=int(input("Enter the salary: ")))
32 net_salary = employee.calculate_net_salary()
33 print(net_salary)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python Debug Console

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53144' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-5.py'
Enter the salary: 52384
41907.2
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c;; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62084' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-5.py'
Enter the salary: 154415
123532.0
```

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization

- o Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

```
if i % 2 == 0:
```

```
total += i
```

```
print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

```
Task-6.py > ...
1 '''Task 6 (Refactoring for Performance Optimization)
2 • Task: Use AI to refactor a performance-heavy loop handling
3 large data.
4 • Focus Areas:
5 o Algorithmic optimization
6 o Use of built-in functions
7 Legacy Code:
8 total = 0
9 for i in range(1, 1000000):
10 if i % 2 == 0:
11 total += i
12 print(total)
13 Expected Outcome:
14 o Optimized logic using mathematical formulas or
15 comprehensions, with time comparison.'''
16
17 # Refactored Code using a Mathematical Formula
18 def sum_of_even_numbers(n):
19     # The sum of the first k even numbers is k * (k + 1)
20     k = n // 2
21     return k * (k + 1)
22 # Example usage
23 n = int(input("Enter the upper limit: "))
24 total = sum_of_even_numbers(n)
25 print(total)
26 # Refactored Code using a List Comprehension
27 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
28 print(total)
29
```

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62107' '---' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-6.py'
Enter the upper limit: 526341
69258712070
249999500000
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '61394' '---' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-6.py'
Enter the upper limit: 58742158
862660310995320
249999500000
```

Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []

def add_item(x):

data.append(x)

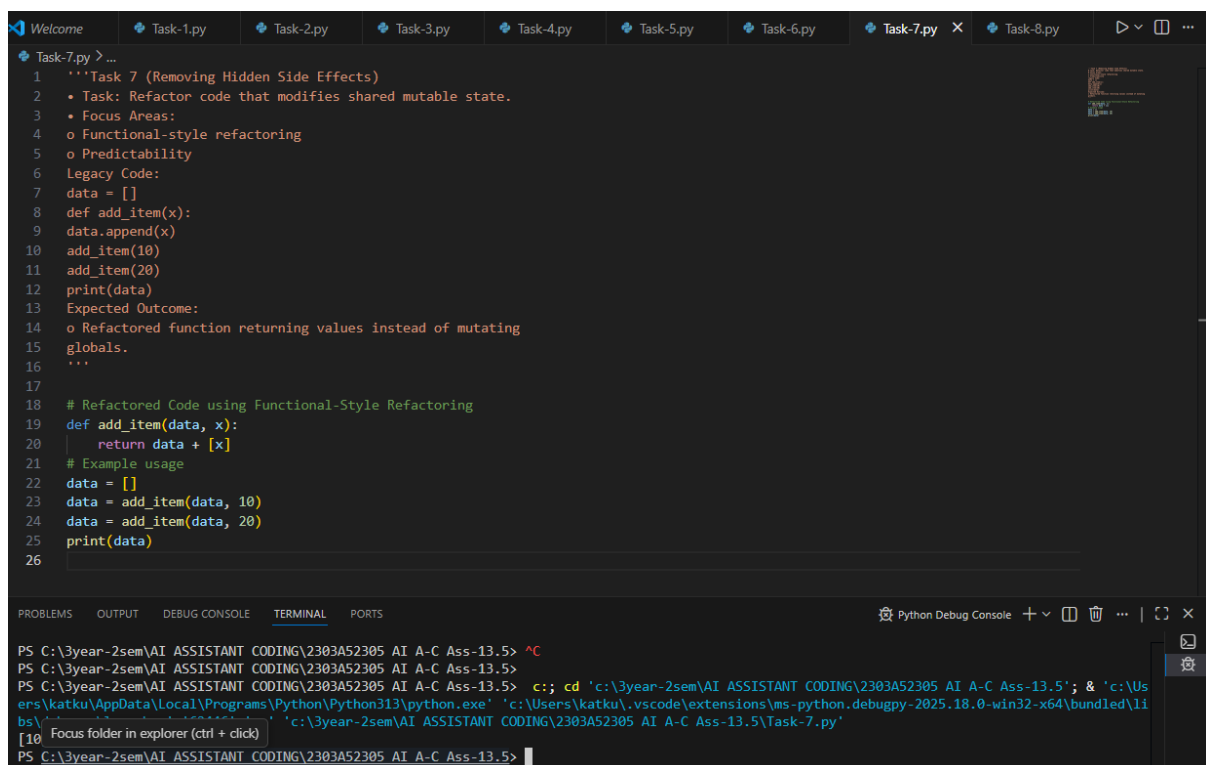
add_item(10)

add_item(20)

print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating globals.



```
1  '''Task 7 (Removing Hidden Side Effects)
2  • Task: Refactor code that modifies shared mutable state.
3  • Focus Areas:
4  o Functional-style refactoring
5  o Predictability
6  Legacy Code:
7  data = []
8  def add_item(x):
9  data.append(x)
10 add_item(10)
11 add_item(20)
12 print(data)
13 Expected Outcome:
14 o Refactored function returning values instead of mutating
15 globals.
16 ...
17
18 # Refactored Code using Functional-Style Refactoring
19 def add_item(data, x):
20     return data + [x]
21 # Example usage
22 data = []
23 data = add_item(data, 10)
24 data = add_item(data, 20)
25 print(data)
26
```

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\lib\python\python.exe' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-7.py'
[10] Focus folder in explorer (ctrl + click)
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
```


Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.

- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
    if any(c.isdigit() for c in password):
```

```
        if any(c.isupper() for c in password):
```

```
            print("Valid Password")
```

```
        else:
```

```
            print("Must contain uppercase")
```

```
    else:
```

```
        print("Must contain digit")
```

```
else:
```

```
    print("Password too short")
```

Expected Outcome:

- o Separate validation functions with clear responsibility

WelcomeTask-1.pyTask-2.pyTask-3.pyTask-4.pyTask-5.pyTask-6.pyTask-7.pyTask-8.py

Task-8.py > contains_uppercase

```
22
23 # Refactored Code using Separate Validation Functions
24 def is_valid_length(password):
25     return len(password) >= 8
26 def contains_digit(password):
27     return any(c.isdigit() for c in password)
28 def contains_uppercase(password):
29     return any(c.isupper() for c in password)
30 def validate_password(password):
31     if not is_valid_length(password):
32         return "Password too short"
33     if not contains_digit(password):
34         return "Must contain digit"
35     if not contains_uppercase(password):
36         return "Must contain uppercase"
37     return "Valid Password"
38 # Example usage
39 password = input("Enter password: ")
40 validation_result = validate_password(password)
41 print(validation_result)
42
```

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

Python Debug Console

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52289' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: Abcd@1234
Valid Password
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52306' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: 1234
Password too short
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> ^C
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5> c:: cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '52323' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5\Task-8.py'
Enter password: Abc1234
Password too short
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-13.5>
```